编译实验优化报告

1. 控制流图（CFG）的构建与维护

遇到的问题

做完死代码删除或者跳转优化后，CFG 的结构经常会对不上。最典型的情况是，后续的遍历算法跑到了一个已经被删掉的基本块里，程序直接报错。还有就是修改了跳转指令的目标后，没有更新前驱和后继的链表，导致后面做活跃变量分析的时候数据流是断的。

解决办法

给基本块的“终结指令”定一个死规则，只要是 Branch、Jump 或者 Return，后面紧跟的指令都当作无效处理。我不建议做增量维护，因为太容易错。最稳妥的办法是，只要做完一步涉及控制流的修改，就强制重新把 CFG 建一遍。另外，维护前驱后继关系时要加个检查，把从入口不可达的块直接清理掉。

2. 支配树与支配关系的计算

遇到的问题

支配关系是 SSA 的基础，如果 CFG 变了但支配信息没更新，后面插 Phi 节点的位置就会全错。刚开始我试图在改 CFG 的同时顺手更新支配树，结果逻辑太复杂，经常出现变量定义覆盖不到使用的情况。

解决办法

严格按顺序来：CFG 稳定 -> 计算支配集 -> 构建支配树。计算支配集的时候，用迭代算法求交集比较稳。要注意边界条件，人为规定“入口块只支配它自己”。一定要等支配关系完全算对了，再开始做 SSA，不然全是无用功。

3. Mem2reg 与 SSA 构造

遇到的问题

这是最容易卡住的地方。主要问题是 Phi 节点漏插，或者重命名的时候版本号搞乱了。比如在 if-else 的汇合点没有正确合并变量值，或者在嵌套循环里，外层的变量值把内层的新定义给覆盖了。

解决办法

插 Phi 节点的时候，必须严格照着“支配边界”算法算出来的结果插，不要自己手动判断。变量重命名时，搞一个基于栈的符号表：进基本块就压栈新变量，出基本块必须弹栈。对于那些没初始化就使用的变量，统一给个 0 或者 undefined，保证 IR 结构完整就行。

4. Phi 节点的消除（SSA 转出）

遇到的问题

把 SSA 转回普通代码时，处理 `a = b; b = a` 这种交换赋值很容易出错。如果直接按顺序生成 move 指令，后一条指令就会用到前一条指令修改过的值，导致结果不对。

解决办法

把 Phi 节点看作是并行的。在拆分的时候，先检查有没有依赖环。如果发现有 `A->B->A` 这种循环依赖，必须引入一个临时变量把环断开。另外，如果从一个有多条出边的块跳转过来，可能需要把边拆分一下，在中间插入一个新块专门用来放这些 copy 指令。

5. 常量传播与死代码删除

遇到的问题

这两个优化容易删过头。比如做常量折叠的时候，把一个带有 IO 输出（printf）的指令因为返回值没被用就给删了。或者把条件恒定的分支删掉后，留下了一堆不可达的基本块，生成汇编的时候标签就乱了。

解决办法

删指令前必须检查副作用，只有纯计算且结果没人用的指令才能删，涉及读写内存和 IO 的绝对不能动。做死代码删除的时候，建议用“标记”法：先把 IO、Store、Return 这些指令标记为“活”的，然后倒着往回找它们的操作数，最后只删那些没被标记过的。

6. 全局值编号（GVN/CSE）

遇到的问题

想合并重复计算，但没处理好内存读写冲突。比如把两次读取 `a[i]` 合并了，但没注意这两次读取中间夹了一个对 `a[j]` 的写入，运行时 `i` 和 `j` 可能相等，导致读到了旧值。

解决办法

内存相关的操作尽量保守。我只对寄存器层面的纯算术指令做激进合并。对于 Load/Store，除非能完全确定访问地址不冲突，否则一旦遇到 Store 指令，就清空当前的 Load 缓存。对于加法乘法这种可交换的操作，算 Hash 之前先把操作数排个序，保证 `a+b` 和 `b+a` 能识别成同一个。

7. 循环优化

遇到的问题

识别循环结构本身就比较麻烦。把循环不变代码提出来以后，经常发现提出来的指令依赖的变量在循环外面还没定义。另外，循环展开之后，Phi 节点的维护很复杂，很容易导致变量引用关系错乱。

解决办法

利用“回边”来识别自然循环，并且强制给每个循环加一个“前置首块”（Preheader）。要把代码外提，只提到这个 Preheader 里，这样能保证逻辑正确。做循环展开时，优先处理定长循环，展开完立刻做一遍死代码删除和 CFG 清理，把多余的跳转和 Phi 节点清干净，不要手动去修补复杂的 Phi 关系。

8. 寄存器分配

遇到的问题

图着色太难写，我用的线性扫描。问题主要出在处理溢出（Spill）的时候。有时候为了把变量存回栈里，插入的 Store 指令用到了一个临时寄存器，但这个寄存器当时正在被别的指令使用，直接把数据覆盖了。

解决办法

在分配前做好活跃变量分析，算出每个基本块的入口和出口有哪些变量是活的。在线性扫描发生溢出时，专门预留一两个特定的寄存器（比如 MIPS 的 k0 或 t8）只用来处理溢出数据的搬运，不让它们参与常规分配，这样最安全。